

Curs 11: Automate FSM (Partea II : Sinteza schemei logice)**Cuprins:**

- Etapele procesului de proiectare a schemei logice a unui automat FSM;
- Sinteza schemei logice pentru automate Moore
 - Cu bistabili de tip D
 - Cu bistabili de tip JK
- Sinteza schemei logice pentru automate Mealy

1. Etapele procesului de proiectare a schemei logice a unui automat FSM

Proiectarea unui automat elementar, pornind de la o descriere a funcționării acestuia în cuvinte (cazul cel mai uzual și totodată cel mai dificil), sau de la o diagramă de tranziție se numește sinteză.

Procesul de sinteză necesită etapelor prezentate mai jos.

♦ Descrierea formală a funcționării automatului

Această etapă este cea mai importantă deoarece trebuie să facă trecerea de la descrierea vagă, uneori ambiguă, a limbajului natural spre o specificație clară a funcționării automatului.

Observații:

- de regulă, sunt cunoscute numărul variabilelor de intrare și al celor de ieșire;
- stărilor automatului le sunt asociate mnemonici cât mai sugestive;
- pe parcursul descrierii pot apare probleme ce nu au fost cuprinse în formularea inițială;
- este posibil ca descrierea corectă să nu rezulte din prima iterație și atunci procesul trebuie reluat;
- rezultatul final al acestei etape este o diagramă de stare sau un tabel de stare ieșire.

♦ Reducerea numărului de stări ale automatului

Din etapa anterioară este posibil ca descrierea formală să conțină un număr mai mare de stări decât cel strict necesar. Reducerea numărului de stări se bazează pe identificarea stărilor echivalente. Două stări ale automatului sunt echivalente, și deci pot fi înlocuite cu una singură, dacă ele nu se pot deosebi prin inspectarea stării curente și a celei următoare. Trebuie remarcat că în procesul de comparare nu participă variabilele de stare.

Se poate demonstra că stările A și B, sunt echivalente dacă sunt îndeplinite următoarele condiții:

- A și B trebuie să conducă la aceleași valori ale variabilelor de ieșire ale automatului, pentru automatele Mealy această condiție trebuie îndeplinită pentru toate variabilele de intrare;
- pentru fiecare combinație de intrare în parte, din A și B trebuie să se ajungă în aceeași stare următoare.

Observații:

- Foarte adesea, în practică, sunt aplicații în care anumite combinații ale variabilelor de intrare nu apar niciodată, caz în care atât ieșirile cât și stările următoare ale automatului nu au nici o semnificație pentru acesta. În tabelul tranzițiilor aceste cazuri se notează cu *don't care* și sunt tratate ca în cazul simplificărilor de funcții binare.
- Două stări binare, A și B, pot fi echivalente chiar dacă nu conduc spre aceleași stări următoare numai cu condiția realizării de echivalențe între stările următoare, "țintite", de stările inițiale A și B.

- Există metode automate de reducere a stărilor însă sunt mai puțin utilizate de proiectanți, deoarece în baza experienței acumulate, aceștia pot face o descriere minimală a funcționării unui automat.
- Nu întotdeauna, un număr redus de stări înseamnă și o implementare mai simplă, există și situații în care creșterea numărului de stări poate simplifica proiectul.

♦ **Asignarea stărilor automatului**

În această etapă, pentru fiecare stare a automatului, stare până acum referită printr-o mnemonică, i se asociază un cod binar unic. Asocierea de coduri binare, pentru fiecare stare a automatului, se poate face în mod arbitrar sau urmărind un anumit criteriu. Deoarece există mai multe posibilități de codare a stărilor, vor exista tot atâtea variante de scheme logice de realizare, echivalente funcțional dar deosebite ca mod de interconectare. Este greu de estimat care schemă logică, dintre cele posibile, este cea mai redusă în complexitate.

Uzual, codarea stărilor se face cel mai adesea arbitrar, sau folosind un criteriu de optimizare cum ar fi: codarea cu variație minimă sau codarea cu dependență redusă față de intrări.

Codificarea cu variație minimă constă în asignarea unor coduri astfel încât trecerea între două stări succesive ale automatului să se facă prin modificarea unui număr cât mai redus de biți ai cuvântului de stare. Această codificare prezintă avantajul că generează expresii simple pentru funcțiile ce calculează stările următoare.

Codificarea cu dependență redusă față de intrări. Funcțiile de tranziție și cele de ieșire depind atât de starea curentă cât și de intrări. Pe căile de tranziție necondiționate nu se testează nici o variabilă de intrare, în schimb pe cele condiționate se poate ajunge la testarea tuturor variabilelor de intrare. Codificare care conduce la expresii ale funcțiilor de tranziție cu o dependență cât mai redusă față de intrări este denumită *codificare cu dependență redusă*.

Observații:

- numărul maxim al stărilor distincte ale unui automat cu n bistabili este egal cu: 2^n .
- pentru stări apropiate din punct de vedere funcțional se recomandă alocarea unor cuvinte de cod care să difere printr-un singur bit (codare cu variație minimă);
- dacă există stări nefolosite se alege cea mai convenabilă codare pentru stările utilizate, nu trebuie să ne limităm strict la codarea binară naturală;
- schimbarea codului unei stări provoacă modificarea întregii scheme logice a automatului;
- acolo unde este posibil, se recomandă separarea din setul variabilelor de stare a biților, sau a grupurilor de biți, cu semnificație precisă referitoare la schimbarea ieșirilor automatului;

♦ **Alcătuirea tabelului de tranziție/ ieșire**

După ce s-au efectuat etapele anterioare, se optează pentru un tip de bistabil, după care se trece la alcătuirea tabelului de tranziție/ieșire. În acest tabel se trec și semnalele de excitație ai bistabililor automatului. După rezolvarea acestei etape, problema se reduce la implementarea unor funcții binare, funcțiile de excitație a bistabililor și funcțiile de ieșire ale automatului.

♦ **Deducerea ecuațiilor funcțiilor de excitație și a celor de ieșire**

Din tabelul rezultat la pasul anterior se extrage forma analitică a funcțiilor de excitație și a celor de ieșire, după care, prin diverse procedee se minimizează aceste expresii în vederea implementării automatului cu un hardware minimal.

♦ **Implementarea automatului**

Rezultatul final al acestei etape este o schemă logică care trebuie să realizeze o funcționare conformă cu diagrama automatului. Spunem o schemă logică deoarece pot fi o multitudine de realizări logice ale aceluiași automat.

3. Sinteza schemei logice pentru automatele de tip Moore

Exemplul 1: Implementarea cu bistabili de tip D

Pentru un automat FSM specificat prin graful de tranziție alăturat se cer:

- tipul automatului, numărul de intrări, numărul de ieșiri, numărul de variabile de stare ;
- tabelul de tranziție a stărilor ;
- o schema logică a automatului folosind doar bistabili D și rețele de porți pentru funcțiile de excitație respectiv ieșire;

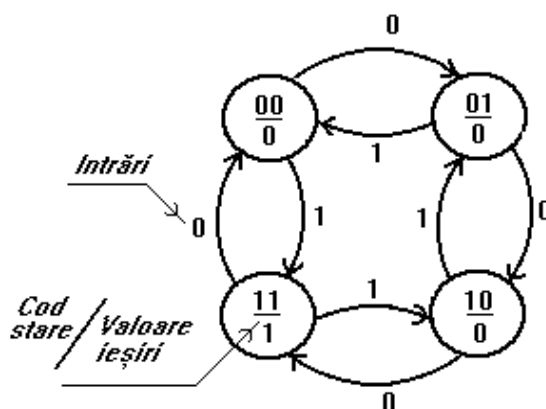


Fig. 1: Graful de tranziție pentru automatul FSM ce trebuie implementat

Rezolvare:

a) Din analiza grafului din enunț, ținând cont de modalitatea de notare acceptată pentru automate, deducem că avem un automat cu următoarele caracteristici:

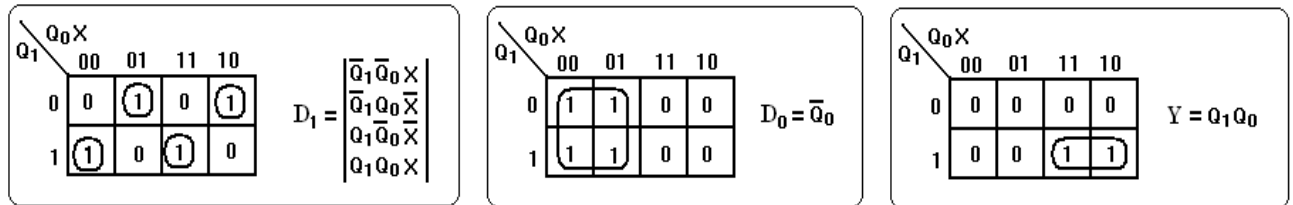
- este de tip Moore deoarece ieșirile depind numai de starea prezentă a automatului, motiv pentru care se trec în interiorul cercului ce reprezintă starea, sub linia de fracție;
- are o singură intrare deoarece pe săgeți este trecută o singură cifră binară pe care o vom nota cu X ;
- are două variabile de stare deoarece în cercul ce reprezintă starea, deasupra liniei de fracție găsim două cifre binare pe care le vom nota Q_1Q_0 ;
- are o singură ieșire deoarece în cercul ce reprezintă starea, sub linia de fracție găsim o singură cifră binare pe care o notăm cu Y_0 ;

b) Tabelul de tranziție a stărilor se obține, destul de ușor din graful de tranziție din enunțul problemei, după cum urmează:

Stare prezentă $Q_1 Q_0$	Intrare X	Stare viitoare $Q_1^+ Q_0^+$	Excitații $D_1 D_0$	Ieșire Y
0 0	0	0 1	0 1	0
0 0	1	1 1	1 1	0
0 1	0	1 0	1 0	0
0 1	1	0 0	0 0	0
1 0	0	1 1	1 1	0
1 0	1	0 1	0 1	0
1 1	0	0 0	0 0	1
1 1	1	1 0	1 0	1

Din graful de tranziție trebuie extrase expresiile algebrice pentru funcțiile de excitație (**D1**, **D0**) și pentru ieșirea **Y** a automatului. Atragem atenția că toate aceste funcții sunt dependente de starea curentă și de intrare și nu depind de starea viitoare.

Diagramele Karnaugh pentru cele trei funcții sunt prezentate mai jos:



Expresia algebrică furnizată de diagramă pentru **D1**, mai poate fi simplificată pe baza altor proprietăți ale algebrei binare, așa cum se prezintă mai jos:

$$D_1 = \begin{vmatrix} \overline{Q_1} \overline{Q_0} X \\ \overline{Q_1} Q_0 \overline{X} \\ Q_1 \overline{Q_0} \overline{X} \\ Q_1 Q_0 X \end{vmatrix} = \begin{vmatrix} \overline{Q_1} \overline{Q_0} X \\ \overline{Q_1} Q_0 \overline{X} \\ Q_1 \overline{Q_0} \overline{X} \\ Q_1 Q_0 X \end{vmatrix} = \begin{vmatrix} \overline{Q_1} & xor(Q_0, X) \\ Q_1 & xnor(Q_0, X) \end{vmatrix} = \begin{vmatrix} \overline{Q_1} & xor(Q_0, X) \\ Q_1 & xor(Q_0, X) \end{vmatrix} = xor[Q_1, xor(Q_0, X)]$$

Pentru D_0 și pentru Y , expresiile furnizate sunt minimale și sunt date de relațiile:

$$D_0 = \overline{Q_0}$$

$$Y = Q_1 Q_0$$

Pe baza celor trei relații se poate desena următoarea schemă logică a automatului:

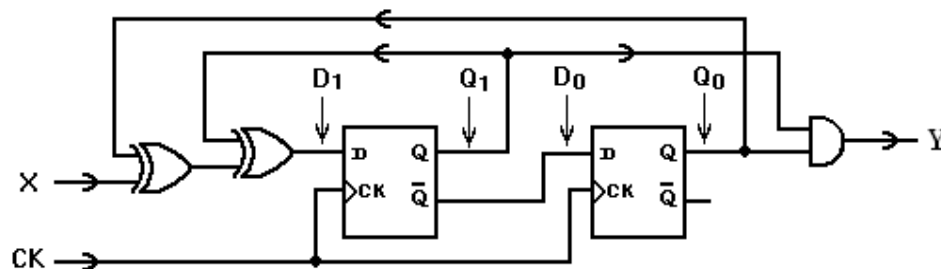


Fig. 2: O schemă logică ce implementează graful de tranziție din figura 1

Exemplul 2: Implementarea cu bistabili de tip JK

Se cere sinteza unei scheme logice pentru graful de tranziție din exemplul anterior (graful din figura 1)

Rezolvare:

Pentru implementarea schemei logice cu bistabili JK este necesară refacerea coloanei cu semnalele de excitație din tabelul de tranziție de la exemplul anterior.

În completarea semnalelor de excitație trebuie să ținem cont de tabelul de tranziție al bistabilului JK așa cum se poate vedea în figura de mai jos:

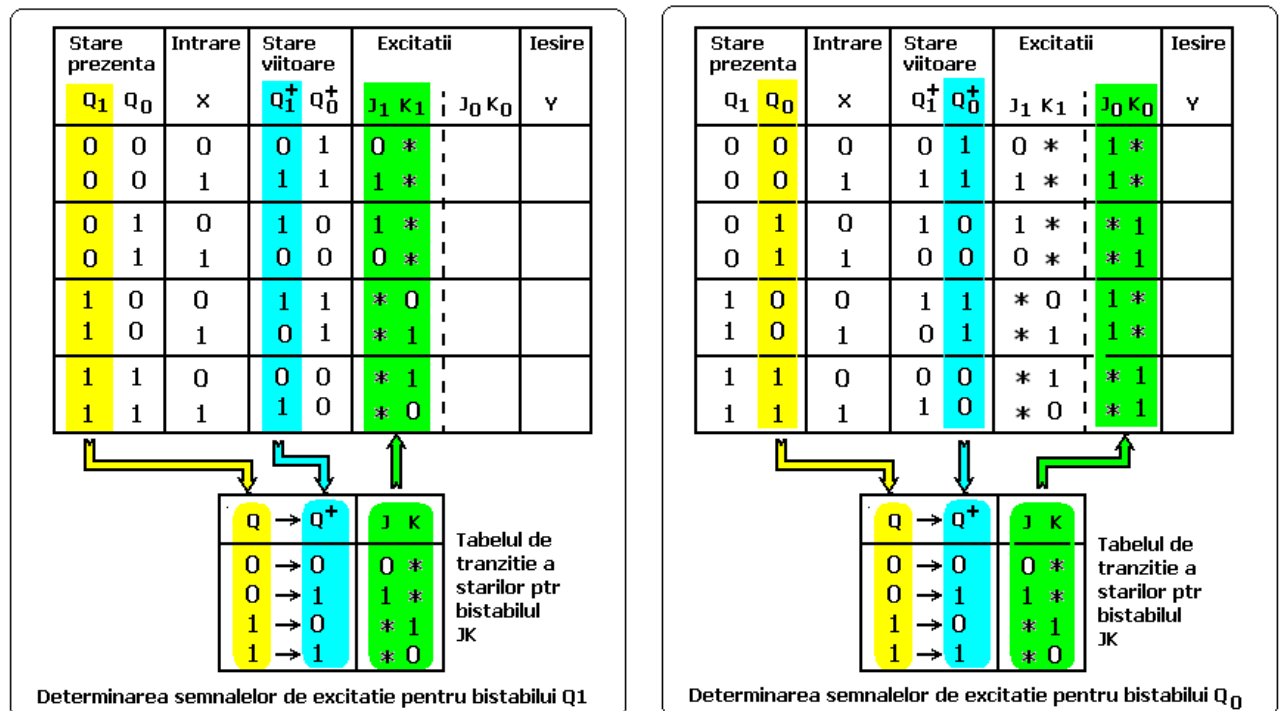
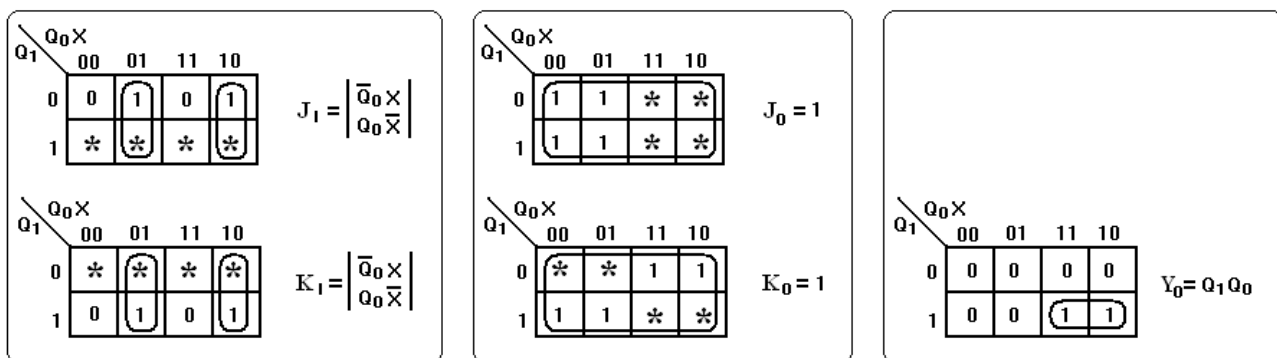


Fig. 3: Construcția tabelului de tranziție a stărilor pt. implementarea cu bistabili JK a grafului din fig.1

După descărcarea valorilor logice pentru ieșirea Y, se obține următoarea formă completă a tabelului de tranziție a stărilor:

Stare prezenta		Intrare x	Stare viitoare		Excitații		Iesire y
Q ₁	Q ₀		Q ₁ ⁺	Q ₀ ⁺	J ₁ K ₁	J ₀ K ₀	
0	0	0	0	1	0 *	1 *	0
0	0	1	1	1	1 *	1 *	0
0	1	0	1	0	1 *	* 1	0
0	1	1	0	0	0 *	* 1	0
1	0	0	1	1	* 0	1 *	0
1	0	1	0	1	* 1	1 *	0
1	1	0	0	0	* 1	* 1	1
1	1	1	1	0	* 0	* 1	1

Determinarea expresiilor algebrice pentru cele 4 semnale de excitație și pentru ieșirea automatului se face cu ajutorul următoarelor diagrame Karnaugh:



Deoarece expresiile algebrice deduse din diagramele anterioare sunt minimale, ele pot fi folosite în desenarea schemei logice a automatului, generând schema ce urmează:

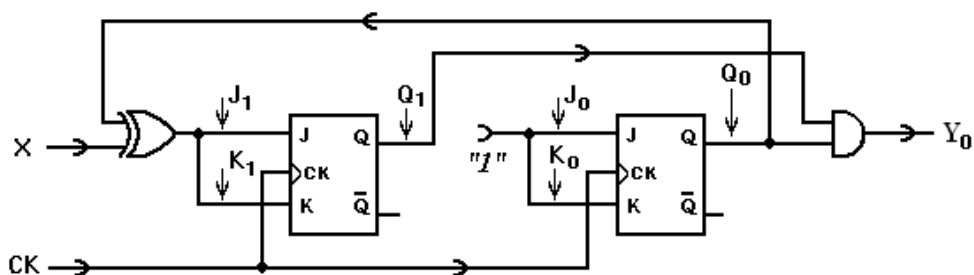


Fig. 4: O schemă logică pentru graful din figura 1, bazată pe bistabili JK

Obs:

Din compararea schemelor din figurile 3 și 4 se constată că schema cu bistabili JK are o complexitate mai mică față de cea realizată cu bistabili D, datorită faptului că bistabilul JK are două intrări pasive iar ecuația funcțională este "mai puternică".

4. Sinteza schemei logice pentru automatele de tip Mealy

Exemplu: Pentru un automat FSM specificat prin graful de tranziție din figura 5 se cer:

- tipul automatului, numărul de intrări, numărul de ieșiri, numărul de variabile de stare ;
- tabelul de tranziție a stărilor ;
- o schema logică a automatului folosind doar bistabili JK și rețele de porți pentru funcțiile de excitație respectiv ieșiri;

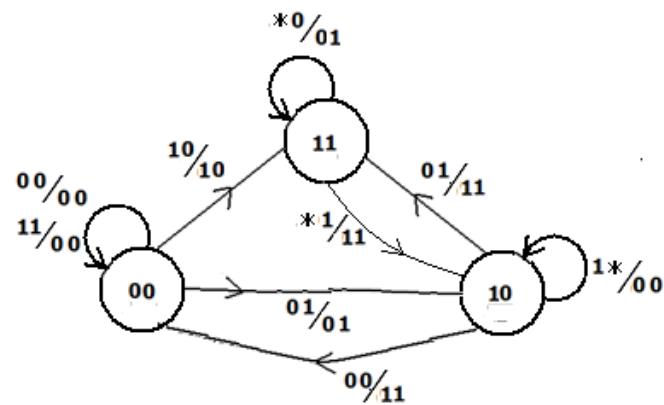


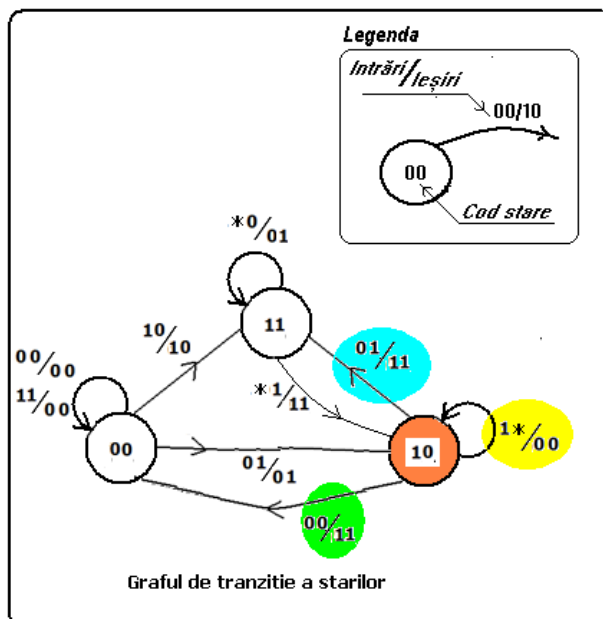
Fig. 5: Graful de tranziție pentru automatul FSM ce trebuie implementat

Rezolvare:

a) Din analiza grafului din enunț, ținând cont de modalitatea de notare acceptată pentru automate, deducem că avem un automat cu următoarele caracteristici:

- este de tip Mealy deoarece ieșirile depind în mod direct de intrări, motiv pentru care se trec pe arce sub linia oblică (/);
- are două intrări deoarece pe săgeți, deasupra semnelui „/” sunt trecute două cifre binare. Notăm aceste intrări prin X_1, X_0 ;
- are două variabile de stare deoarece în cercul ce reprezintă starea găsim două cifre binare Q_1Q_0 ;
- are două ieșiri deoarece, pe arce, sub semnul „/” sunt trecute două cifre binare. Notăm aceste ieșiri prin Y_1, Y_0 ;

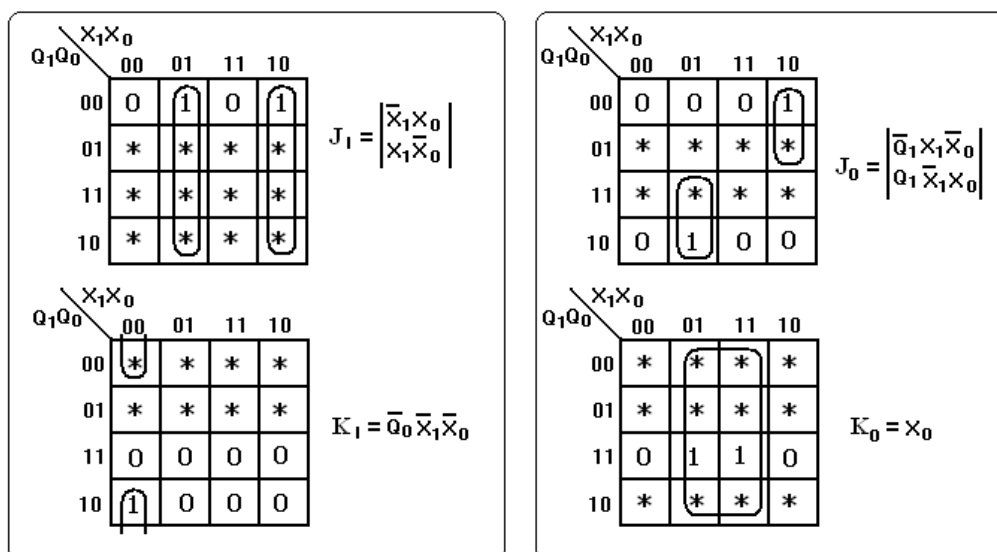
b) Tabelul de tranziție a stărilor se obține folosind modul de lucru din figura de mai jos:



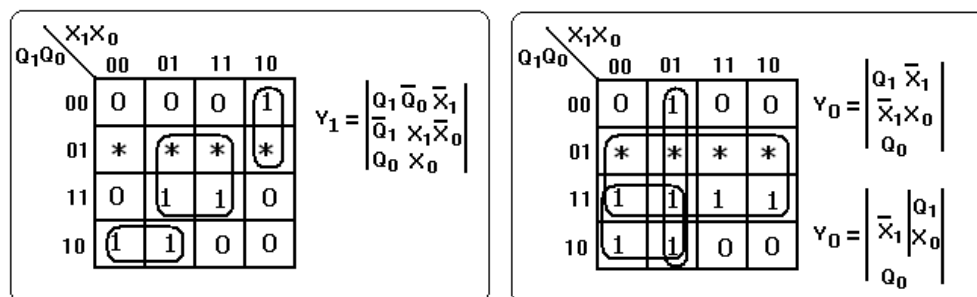
Stare prezenta $Q_1 Q_0$	Intrari $x_1 x_0$	Stare viitoare $Q_1^+ Q_0^+$	Excitații		Ieșiri
			$J_1 K_1$	$J_0 K_0$	$Y_1 Y_0$
0 0	0 0	0 0	0 *	0 *	0 0
0 0	0 1	1 0	1 *	0 *	0 1
0 0	1 0	1 1	1 *	1 *	1 0
0 0	1 1	0 0	0 *	0 *	0 0
0 1	0 0	* *	* *	* *	* *
0 1	0 1	* *	* *	* *	* *
0 1	1 0	* *	* *	* *	* *
0 1	1 1	* *	* *	* *	* *
1 0	0 0	0 0	* 1	0 *	1 1
1 0	0 1	1 1	* 0	1 *	1 1
1 0	1 0	1 0	* 0	0 *	0 0
1 0	1 1	1 0	* 0	0 *	0 0
1 1	0 0	1 1	* 0	* 0	0 1
1 1	0 1	1 0	* 0	* 1	1 1
1 1	1 0	1 1	* 0	* 0	0 1
1 1	1 1	1 0	* 0	* 1	1 1

Tabelul de tranziție a stărilor

Diagramele Karnaugh pentru cele patru semnale de excitație sunt prezentate mai jos:



Pentru determinarea funcțiilor de ieșire se folosesc următoarele diagrame Karnaugh:



Pe baza relațiilor determinate cu ajutorul diagramelor anterioare se obține următoarea schemă logică a automatului

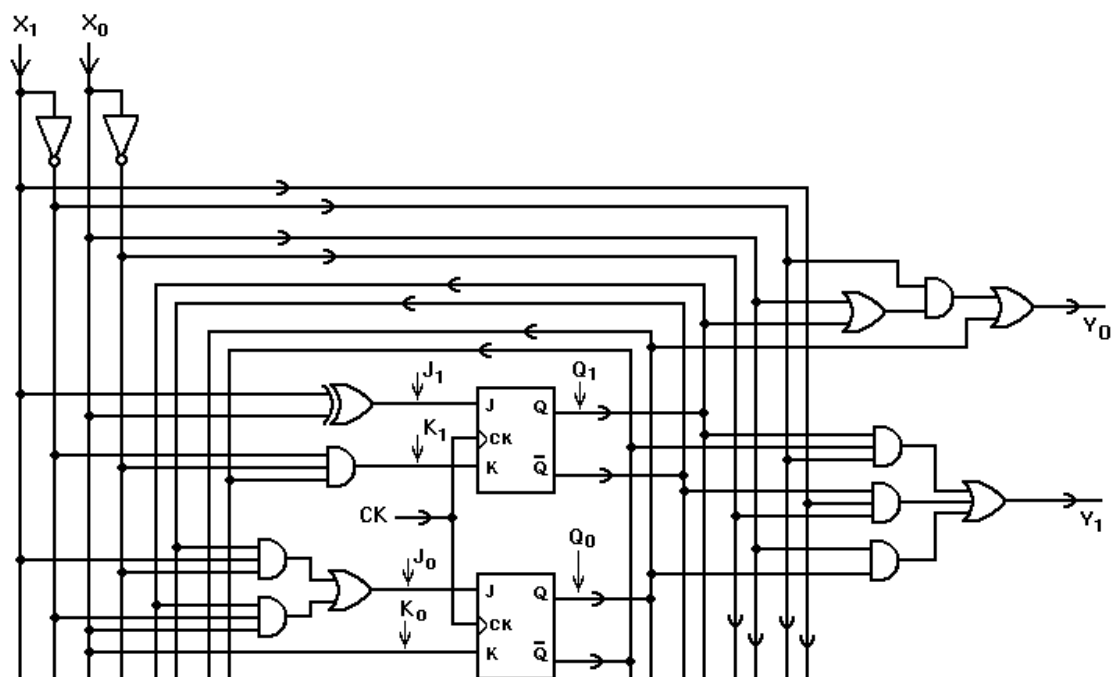


Fig. 6: O implementare cu bistabili JK pentru graful de tranziție din figura 5